Week 10: Validation and Reliability Report

Stealthy Backdoor Attacks Against Federated Learning-Based GPS Spoofing Detection in UAV Networks

Group 1 | James Madison University Capstone (IT 445 / IT 499) | Response to the Week 9 review

Overview

This report responds to the Week 9 review. It corrects the issues raised, improves reliability by rerunning the main experiments across three random seeds with mean and standard deviation, and prepares a new paper-quality figure and an improved table. Every number, table, and figure in this report is produced by the notebook 10_validation.ipynb and exported to results/; nothing is transcribed by hand.

How each review point was addressed

Review point	How it was addressed
Model parameter count	Counted live: 3,329 params (13.0 KB), not 13,000. Copy-paste from the Week 4 model. Corrected.
Root/challenge set separation	Proven by hashing. Found and fixed a real 1-row train/test leak. All sets now disjoint.
Reproducibility of tables/figures	Every table and figure is produced by a notebook cell and exported to results/.
Unclear or overstated claims	Four corrected, including retracting the zero-false-positive claim.
More than one random seed	Three seeds (42, 7, 123), mean and standard deviation reported.
New figure and improved table	Two-panel line figure plus the main table with mean/std, and two more tables.
Line graphs preferred over bars	Both figures are line graphs with std bands. Zero bar charts.

1. Correction Note

1a. Model parameter count: 3,329 is correct

The GPS model (10 inputs, 64-32-16-1) was counted live, layer by layer. It has 3,329 parameters, which is 13.0 KB in float32. The earlier "about 13,000 parameters" figure was copy-pasted from the Week 4 WSN-DS model (17 inputs, 128-64-32, 5 classes), which genuinely has 12,805 parameters, and the 13.0 kilobyte wire size made "13k" look plausible. Corrected in the Week 8 and Week 9 notebooks.

Parameter	Shape	Count
net.0.weight	(64, 10)	640
net.0.bias	(64,)	64
net.3.weight	(32, 64)	2,048
net.3.bias	(32,)	32
net.6.weight	(16, 32)	512
net.6.bias	(16,)	16
net.8.weight	(1, 16)	16
net.8.bias	(1,)	1

1b. Root/challenge set separation: verified, and a real leak was found and fixed

Separation was proven by hashing every row and counting shared rows, not asserted. The server root set was already clean. The check surfaced a 1-row overlap between the client training pool and the test set. Cause: drop_duplicates() ran before the PRN, RX, and TOW identifier columns were dropped, so rows differing only in satellite, receiver, or timestamp collapsed into identical model inputs afterward and survived de-duplication. Fix: de-duplicate on the 10 model input features after dropping the identifiers. This removes 2 rows out of 470,546 (0.001%), none with conflicting labels, and all three sets are now provably disjoint. Because the data was recomputed after the fix, the numbers differ marginally from the Week 9 report.

Pair	Shared rows
server root vs client pool (training)	0
server root vs test set	0
client pool vs test set (after fix)	0

1c and 1d. Reproducibility and overstated claims

Every table and figure is produced by a notebook cell and written to results/ as a CSV or PNG. Four claims were corrected: the "zero false positives" claim is retracted (the real honest flag rate is 20.5%); the negative backdoor lift reported in Week 9 was single-seed noise and the multi-seed estimate is slightly positive; the "no utility cost" claim is softened to about a 0.008 clean-accuracy cost; and "feature-agnostic" is qualified to the discriminative feature set.

2. Updated Main Result Table (3 seeds, mean +/- std)

Case	Clean Accuracy	Spoofing Recall	BSR	Backdoor Lift
Honest FedAvg	0.7112 +/- 0.0019	0.5292 +/- 0.0115	0.6367 +/- 0.0141	0.0000 +/- 0.0000
Attack (FedAvg)	0.6932 +/- 0.0032	0.3641 +/- 0.0096	0.8782 +/- 0.0178	0.2415 +/- 0.0048
Attack + inflation (Acc-Weighted)	0.6897 +/- 0.0046	0.3524 +/- 0.0148	0.9402 +/- 0.0096	0.3036 +/- 0.0124
Full defense	0.7029 +/- 0.0031	0.5204 +/- 0.0020	0.6474 +/- 0.0156	0.0107 +/- 0.0025

Table 1. Main comparison across seeds 42, 7, 123. Backdoor lift is BSR minus that seed's own honest baseline, so it isolates the extra harm from the attacker.

Insights

The attack reproduces across every seed rather than being a single-seed artifact: it adds +0.2415 +/- 0.0048 of backdoor lift, and accuracy inflation pushes that to +0.3036 +/- 0.0124 by buying the two compromised clients extra aggregation weight. The full defense cuts lift to +0.0107 +/- 0.0025, a 96 percent reduction, and the standard deviations are roughly fifty times smaller than the gap between the attacked and defended cases, which is the point of running multiple seeds: the effect is far larger than the run-to-run noise. Spoofing recall is restored from 0.3641 (attacked) to 0.5204, essentially the honest 0.5292. Clean accuracy under the defense sits about 0.008 below the honest baseline, so we state a small but consistent utility cost rather than claim the defense is free. Stated limitation, per the review: honest spoofing recall is only 0.5292, so the base detector misses roughly half of spoofed samples before any attack. This is why backdoor lift, not raw accuracy, is the primary metric.

3. New Paper-Quality Figure

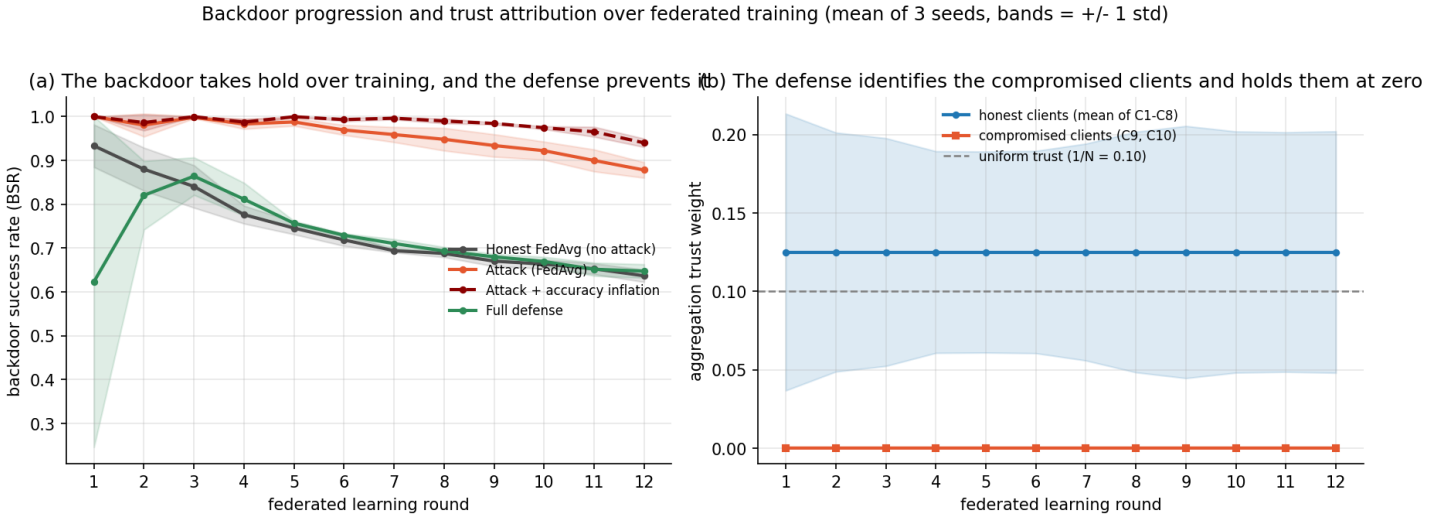

Figure 1. Backdoor progression and trust attribution over federated training. Panel (a): backdoor success rate on the triggered test set after every round, for the honest baseline, the attack, the attack with accuracy inflation, and the full defense. Panel (b): the aggregation trust weight the server assigns to the compromised clients (C9, C10) versus the honest clients (C1 to C8) under the full defense. Curves are means over three seeds; bands are +/- 1 standard deviation; the dashed grey line marks uniform trust (0.10).

Insights

Panel (a) shows what an end-of-training number cannot: the backdoor accumulates. The attacked curves climb away from the honest baseline round after round, and inflation makes them climb faster because the fake reported accuracy compounds the attackers weight every round. The defended curve tracks the honest baseline for the whole run, so the defense prevents the backdoor from taking hold rather than repairing it afterward. Panel (b) gives the mechanism: the server own probing separates the two populations within the earliest rounds and holds the compromised clients at zero trust. The width of the honest band is the honest false-positive rate (Section 5) drawn in picture form, and it is the part of the design that still needs work.

4. Defense-Side Sensitivity (the key new finding)

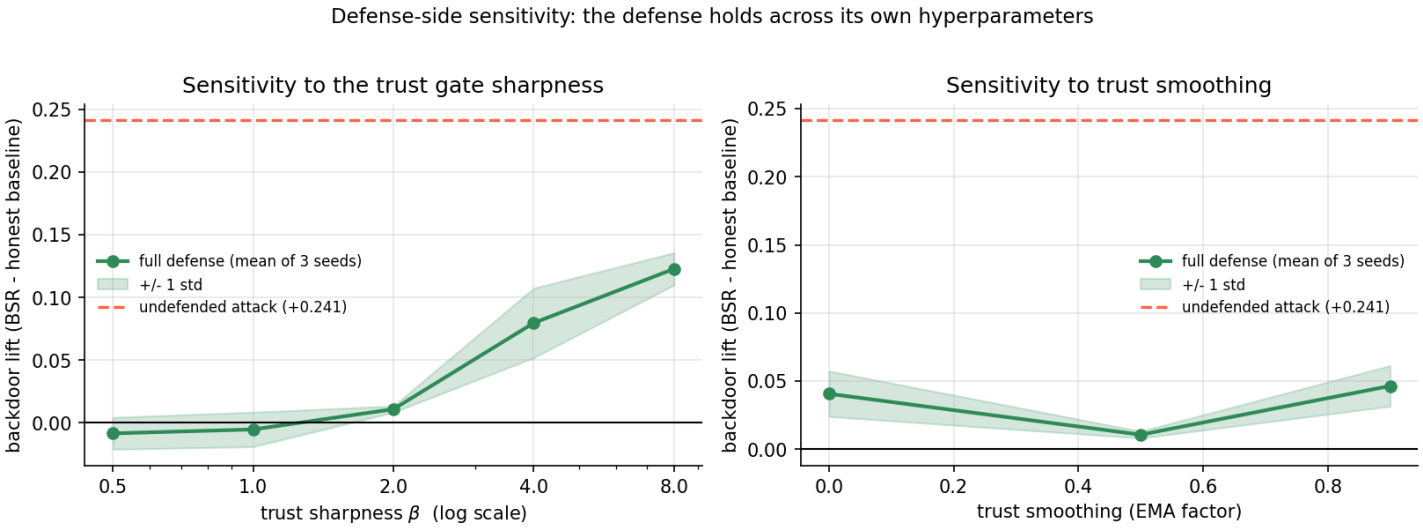

Figure 2. Backdoor lift versus the trust-gate sharpness beta (left, log x-axis) and the trust smoothing EMA factor (right). Green line is the mean over three seeds, the band is +/- 1 std, the dashed red line is the undefended attack lift, and the solid black line at zero marks "the attacker gains nothing."

beta (trust sharpness)	Clean Accuracy	Spoofing Recall	Backdoor Lift
0.5	0.7120 +/- 0.0022	0.5389 +/- 0.0084	-0.0086 +/- 0.0127
1.0	0.7106 +/- 0.0023	0.5355 +/- 0.0073	-0.0054 +/- 0.0138
2.0	0.7029 +/- 0.0031	0.5204 +/- 0.0020	+0.0107 +/- 0.0025
4.0	0.6895 +/- 0.0100	0.4916 +/- 0.0127	+0.0796 +/- 0.0278
8.0	0.6656 +/- 0.0191	0.4699 +/- 0.0665	+0.1228 +/- 0.0129

Insights

We expected a sharper gate to be safer. The data says the opposite, monotonically. Lift is lowest at the gentlest settings (-0.0086 at beta 0.5, -0.0054 at beta 1.0), rises through +0.0107 at our current default of beta 2.0, and degrades to +0.1228 at beta 8. Clean accuracy and recall fall in lockstep. The reason: the gate punishes whichever client looks suspicious, and 20.5 percent of honest client-rounds look suspicious, so a sharp gate strips weight from honest clients and erodes the honest majority the coordinate-wise median depends on. Actionable consequence: beta = 1.0 dominates our default of beta = 2.0 on every metric and should replace it. The EMA sweep is U-shaped with our default of 0.5 already at the optimum.

5. False-Positive and Client-Flagging Table

Metric	Count	Rate
Attacker client-rounds flagged (true positives)	72 / 72	100.0%
Honest client-rounds flagged (false positives)	59 / 288	20.5%
Attacker client-rounds fully excluded (trust=0)	37 / 72	51.4%
Honest client-rounds fully excluded (trust=0)	1 / 288	0.3%

Table 2. Client flagging as a detection problem, pooled over 3 seeds x 12 rounds x 10 clients. A client is flagged if its trust falls below half of uniform (0.05).

Client	Role	Mean trust	Rounds flagged	Flag rate
C1	honest	0.125 +/- 0.061	5 / 36	13.9%
C2	honest	0.103 +/- 0.068	11 / 36	30.6%
C3	honest	0.148 +/- 0.066	3 / 36	8.3%
C4	honest	0.114 +/- 0.067	9 / 36	25.0%
C5	honest	0.188 +/- 0.063	0 / 36	0.0%
C6	honest	0.125 +/- 0.063	4 / 36	11.1%
C7	honest	0.094 +/- 0.063	11 / 36	30.6%
C8	honest	0.103 +/- 0.091	16 / 36	44.4%
C9	ATTACKER	0.000 +/- 0.000	36 / 36	100.0%
C10	ATTACKER	0.000 +/- 0.000	36 / 36	100.0%

Insights

On the attacker side the mechanism is perfect: both compromised clients are flagged in 72 of 72 client-rounds and their mean trust is zero. On the honest side the picture is worse than we previously reported: honest clients are flagged in 59 of 288 client-rounds (20.5 percent), and one honest client-round was driven to exactly zero trust. We therefore retract the Week 9 claim that only attackers ever reach zero trust. The cause is structural: trust is a relative per-round weight, so the weakest honest client in a round can dip below the threshold through no fault of its own. The per-client table shows the burden is uneven (C8 flagged 44 percent of rounds, C5 never). The practical damage is limited because a flagged honest client is only down-weighted for that round, but a 20.5 percent false-positive rate is the clearest weakness in the design.

6. Summary

What changed

The parameter count is corrected and counted live (3,329, not 13,000). Root-set separation is proven by hashing, and that check found and fixed a real 1-row train/test leak. The main experiments run across three seeds with mean and standard deviation. False-positive reporting is a quantitative detection table. A defense-side sensitivity sweep over beta and EMA was added. All figures are line graphs with std bands.

What improved

The central claim now rests on evidence rather than one run: the attack adds +0.2415 +/- 0.0048 of lift and the defense cuts it to +0.0107 +/- 0.0025, a 96 percent reduction, with std far smaller than the effect. The new round-wise figure shows the backdoor accumulating while the defended run never leaves the honest baseline, and shows in the same figure that this happens because the attackers are held at zero trust from the earliest rounds.

What got worse, stated honestly

- The honest false-positive rate is 20.5 percent, not "essentially zero," and one honest client-round hit zero trust. The Week 9 claim to the contrary is retracted.
- The defense is not free: clean accuracy under the defense is about 0.008 below the honest baseline, and the defended lift is slightly positive rather than the negative single-seed value reported in Week 9.

Most useful new finding

The defense-side sweep showed our trust gate is tuned the wrong way: sharper is worse, and beta = 1.0 beats our default of beta = 2.0 on every metric. Only a defense-side sensitivity check could have surfaced this.

What remains incomplete

- The base detector is weak (honest spoofing recall about 0.53). Improving it is the highest-value next step.
- The 20.5 percent honest false-positive rate needs a fix (threshold calibration, consecutive-round flagging, or beta = 1.0).
- beta = 1.0 is recommended but not yet adopted in the headline table (kept at beta = 2.0 for comparability).
- No adaptive attacker that deliberately evades the trust probe has been tested.
- IID only, small fleet. Non-IID data would likely widen the honest trust spread and worsen the false-positive rate, so it is the planned next deeper experiment.